

2.0 Introduction to Linear Time-Invariant Systems

A system is called an LTI system if it satisfies:

1. Linearity
2. Time Invariance

LTI systems are important because they can be completely characterized using their impulse response.

In this notebook we will learn LTI systems primarily through simulations and examples.

Problem 1

Consider the system:

$$y(t) = 2x(t)$$

Verify whether the system satisfies linearity.

```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0,10,1000)

x1 = np.sin(t)
x2 = np.cos(t)

a = 2
b = 3

def system(x):
    return 2*x

lhs = system(a*x1+b*x2)
rhs = a*system(x1)+b*system(x2)
error = np.max(np.abs(lhs-rhs))
print("Maximum Error =",error)
Maximum Error = 0.0
```

Observation

The error is approximately zero.

Hence the system satisfies the superposition principle and is linear.

Problem 2

Consider the system:

$$y(t) = x^2(t)$$

Verify whether the system is linear.

```
def nonlinear_system(x):  
    return x**2  
  
lhs = nonlinear_system(a*x1+b*x2)  
rhs = a*nonlinear_system(x1)+b*nonlinear_system(x2)  
error = np.max(np.abs(lhs-rhs))  
print("Maximum Error =",error)  
Maximum Error = 10.324547198850393
```

Observation

The error is not zero.

Hence the system is nonlinear.

2.1 Discrete-Time LTI Systems

For an LTI system:

$$y[n] = x[n] * h[n]$$

where:

- $x[n]$ = input signal
- $h[n]$ = impulse response
- $y[n]$ = output signal

The operation $*$ is called convolution.

Problem 3

Input Signal:

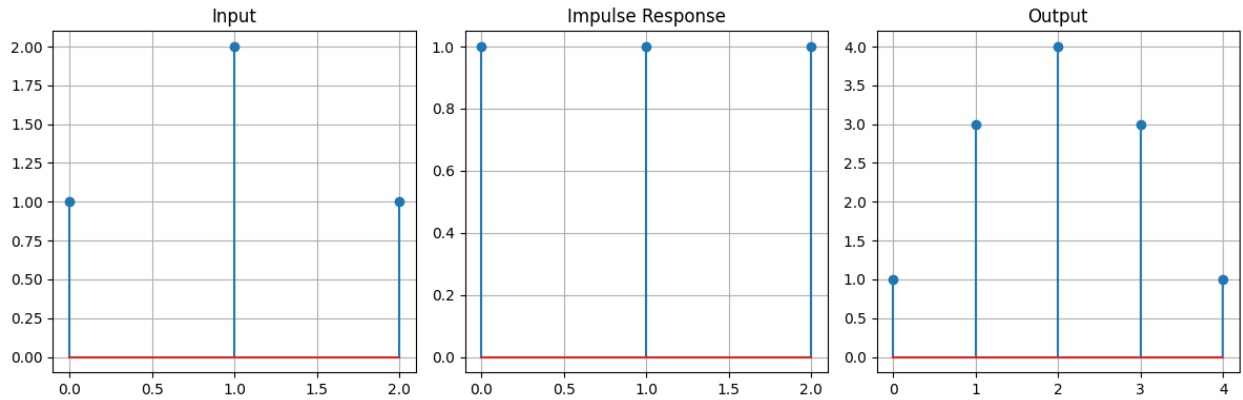
$$x[n] = [1, 2, 1]$$

Impulse Response:

$$h[n] = [1, 1, 1]$$

Compute the output.

```
x = np.array([1,2,1])
h = np.array([1,1,1])
y = np.convolve(x,h)
print(y)
[1 3 4 3 1]
plt.figure(figsize=(12,4))
plt.subplot(131)
plt.stem(x)
plt.title("Input")
plt.grid()
plt.subplot(132)
plt.stem(h)
plt.title("Impulse Response")
plt.grid()
plt.subplot(133)
plt.stem(y)
plt.title("Output")
plt.grid()
plt.tight_layout()
plt.show()
```



Observation

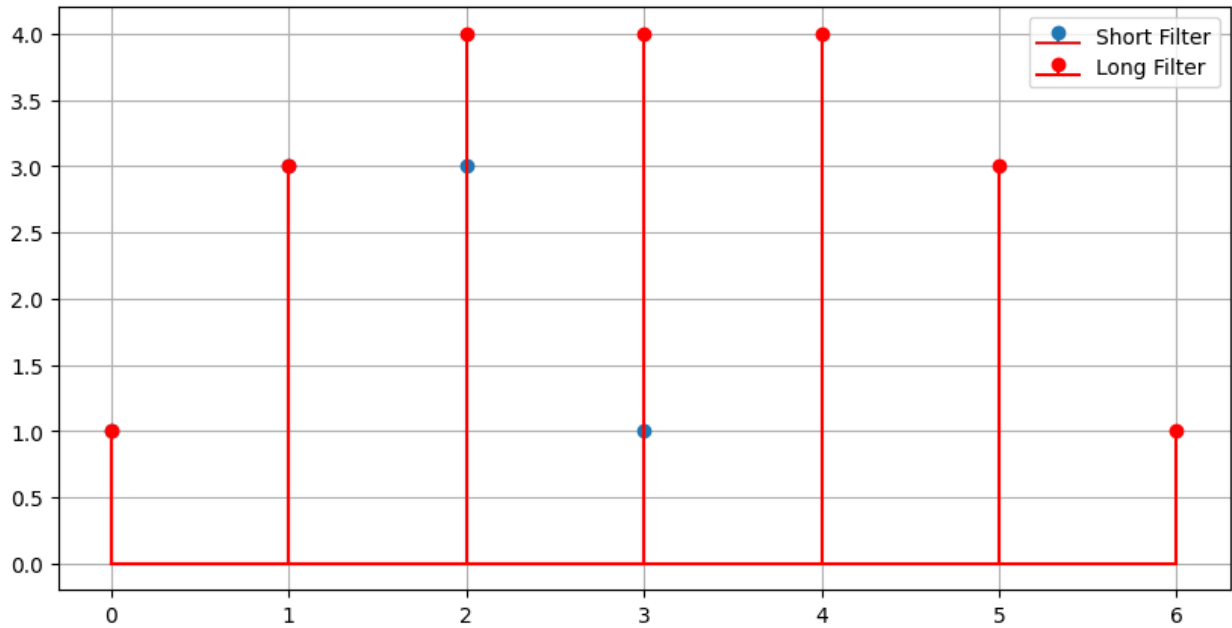
The output contains contributions from every input sample.

Convolution acts as a weighted accumulation process.

Problem 4

Investigate how output changes when the impulse response length increases.

```
h1 = np.array([1,1])
h2 = np.array([1,1,1,1,1])
y1 = np.convolve(x,h1)
y2 = np.convolve(x,h2)
plt.figure(figsize=(10,5))
plt.stem(y1,label="Short Filter")
plt.stem(y2,linfmt='r-',markerfmt='ro',
        baselfmt='r-',label="Long Filter")
plt.legend()
plt.grid()
plt.show()
```



Observation

Increasing filter length produces a smoother output.

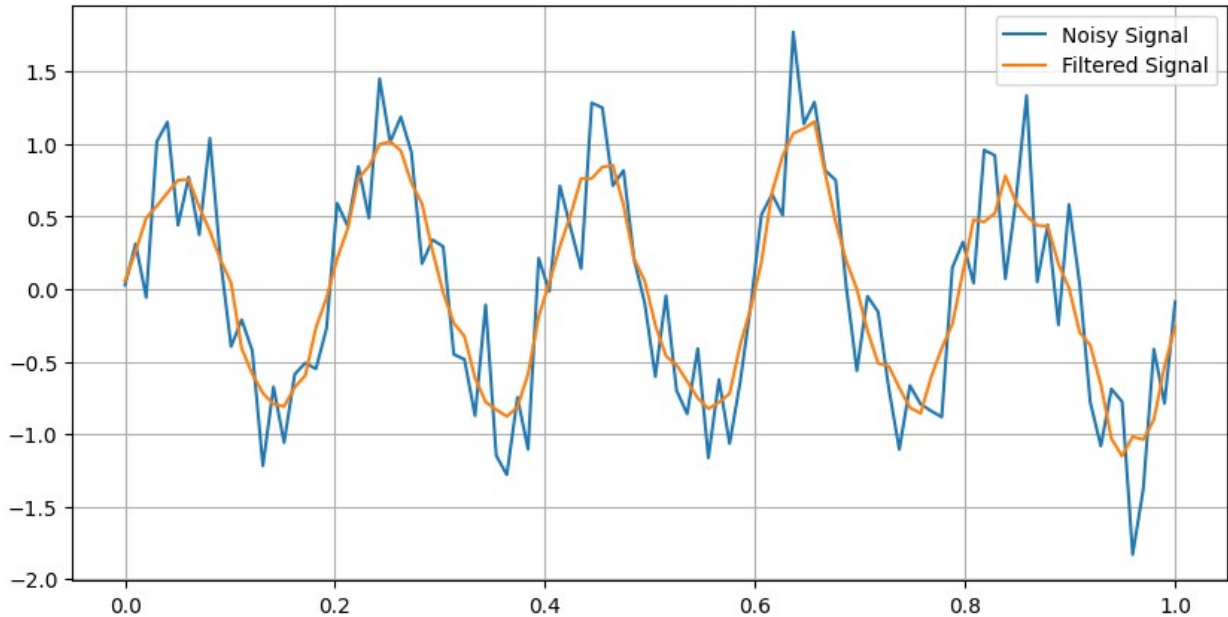
Longer impulse responses influence more samples.

Problem 5

Apply a moving average filter to a noisy signal.

```
t = np.linspace(0,1,100)
signal = np.sin(2*np.pi*5*t)
noise = 0.4*np.random.randn(100)
x = signal + noise
h = np.ones(5)/5
y = np.convolve(x,h,mode='same')
plt.figure(figsize=(10,5))
plt.plot(t,x,label="Noisy Signal")
plt.plot(t,y,label="Filtered Signal")
plt.legend()
```

```
plt.grid()  
plt.show()
```



Observation

The moving average filter reduces high-frequency noise.

This demonstrates a practical application of convolution.

2.2 Continuous-Time Convolution

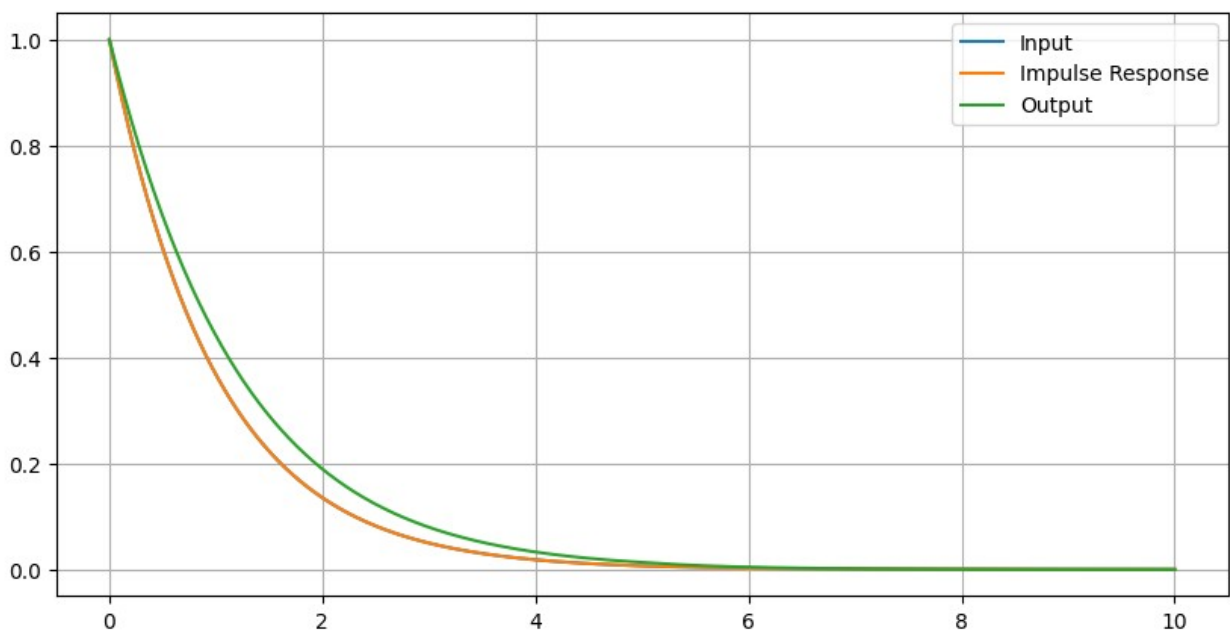
Continuous-time convolution is given by:

$$y(t) = \int_{-\infty}^{\infty} x(\tau)h(t-\tau)d\tau$$

We approximate this numerically.

```
t = np.linspace(0,10,1000)  
x = np.exp(-t)  
h = np.exp(-t)  
y = np.convolve(x,h, 'same')
```

```
plt.figure(figsize=(10,5))
plt.plot(t,x,label="Input")
plt.plot(t,h,label="Impulse Response")
plt.plot(t,y/np.max(y),label="Output")
plt.legend()
plt.grid()
plt.show()
```



Observation

The output is smoother and broader than either input signal.

Continuous-time convolution accumulates overlap between signals.

Section 2.3 — Properties of LTI Systems

Problem 7- Commutative Property

Verify:

$$x * h = h * x$$

```
x = np.array([1,2,1])
h = np.array([1,1,1])
y1 = np.convolve(x,h)
y2 = np.convolve(h,x)
print(np.max(np.abs(y1-y2)))
0
```

Observation

The error is zero.

Convolution is commutative.

Problem 8 — Associative Property

```
x = np.array([1,2])
h1 = np.array([1,1])
h2 = np.array([1,-1])
lhs = np.convolve(np.convolve(x,h1),h2)
rhs = np.convolve(x,np.convolve(h1,h2))
print(np.max(np.abs(lhs-rhs)))
0
```

Observation

The error is zero.

Convolution is associative.

Section 2.4 — Difference Equations

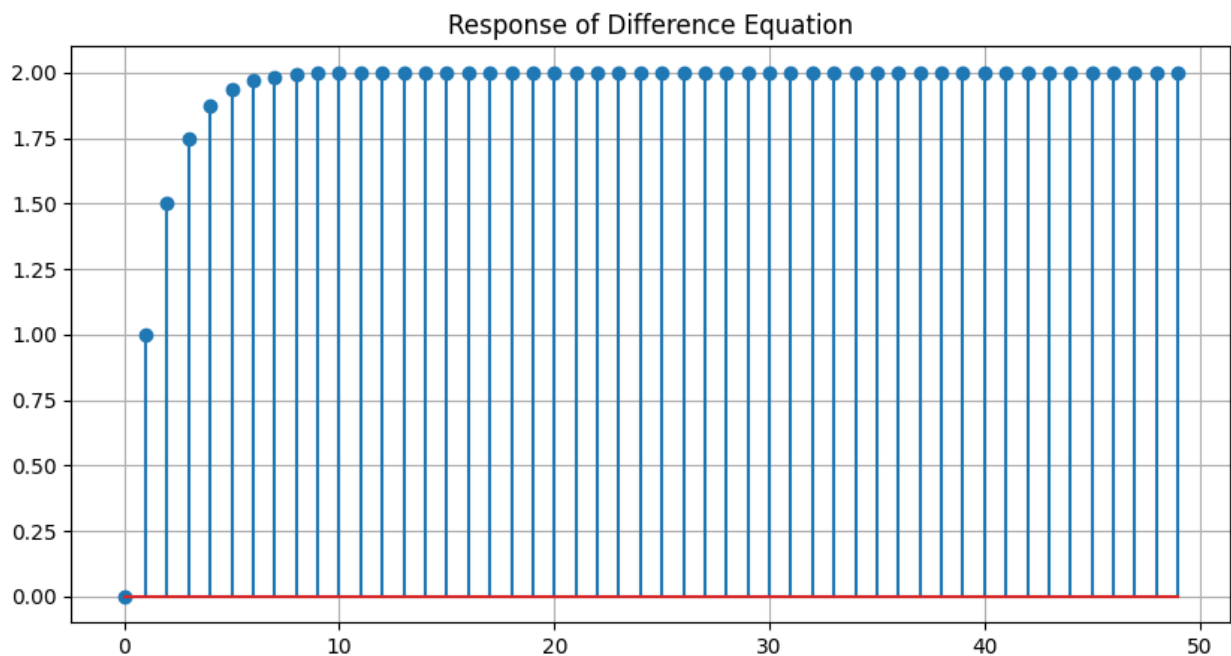
Problem 9

Consider:

$$y[n] = 0.5y[n-1] + x[n]$$

Find the output for a unit-step input.

```
N = 50
x = np.ones(N)
y = np.zeros(N)
for n in range(1,N):
    y[n] = 0.5*y[n-1] + x[n]
plt.figure(figsize=(10,5))
plt.stem(y)
plt.title("Response of Difference Equation")
plt.grid()
plt.show()
```



Observation

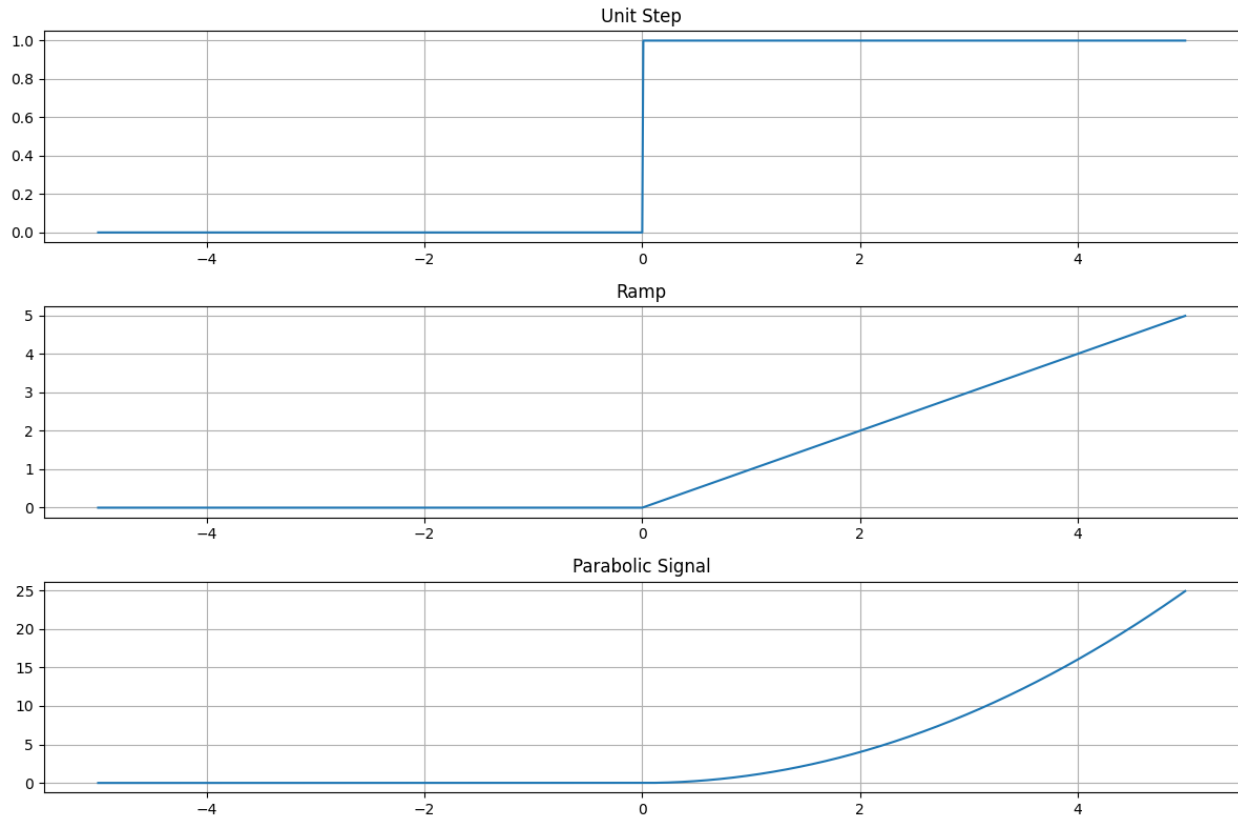
The output gradually reaches a steady-state value.

The system possesses memory because present output depends on past output.

Section 2.5 — Singularity Functions

Problem 10 — Compare Basic Signals

```
t = np.arange(-5,5,0.01)
u = np.where(t>=0,1,0)
r = np.where(t>=0,t,0)
p = np.where(t>=0,t**2,0)
plt.figure(figsize=(12,8))
plt.subplot(311)
plt.plot(t,u)
plt.title("Unit Step")
plt.grid()
plt.subplot(312)
plt.plot(t,r)
plt.title("Ramp")
plt.grid()
plt.subplot(313)
plt.plot(t,p)
plt.title("Parabolic Signal")
plt.grid()
plt.tight_layout()
plt.show()
```



Observation

Step, ramp and parabolic signals are important singularity functions used in system analysis.

Problem 11

Investigate how changing the impulse response changes the output signal.

Input:

$$x[n] = [1, 2, 3, 2, 1]$$

Try three different impulse responses and compare the outputs.

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([1, 2, 3, 2, 1])

h1 = np.array([1])
h2 = np.array([1, 1, 1])
h3 = np.array([1, -1])

y1 = np.convolve(x, h1)
```

```

y2 = np.convolve(x,h2)
y3 = np.convolve(x,h3)

plt.figure(figsize=(12,8))

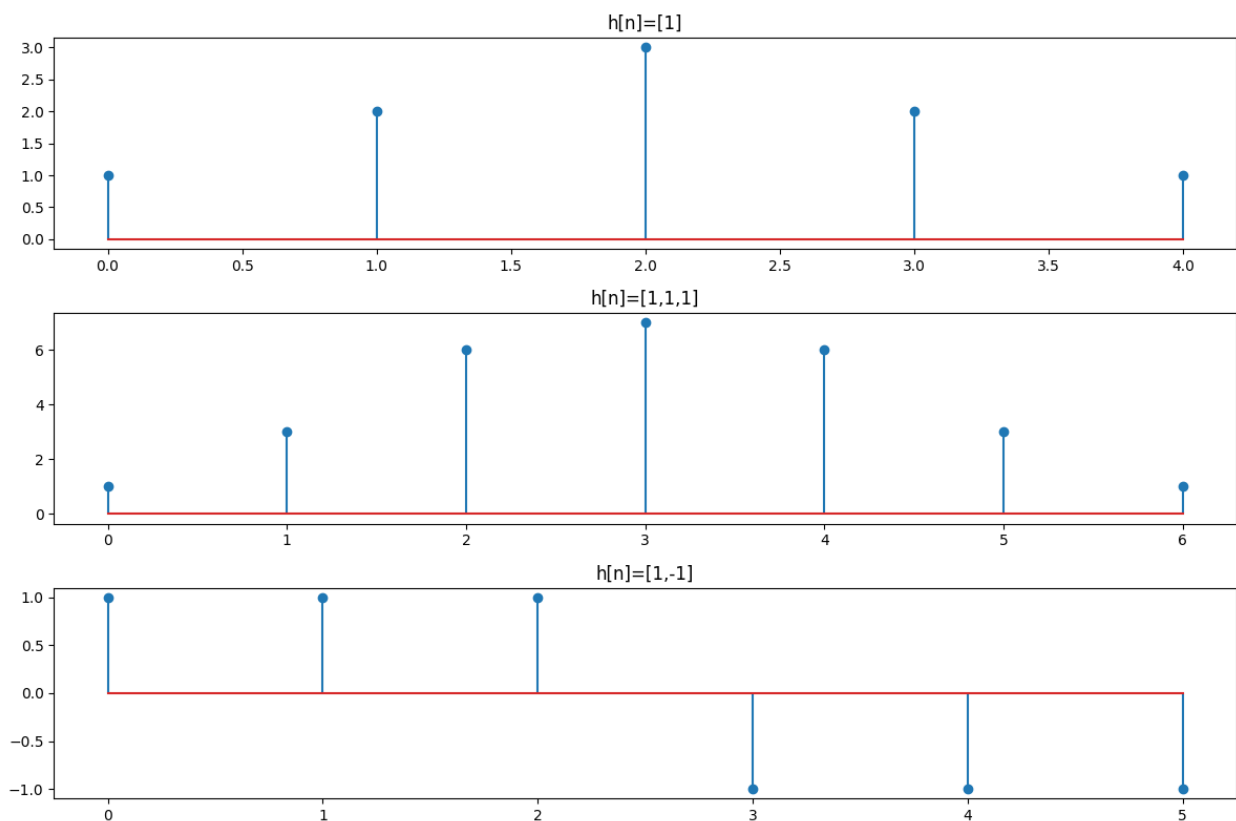
plt.subplot(3,1,1)
plt.stem(y1)
plt.title("h[n]=[1]")

plt.subplot(3,1,2)
plt.stem(y2)
plt.title("h[n]=[1,1,1]")

plt.subplot(3,1,3)
plt.stem(y3)
plt.title("h[n]=[1,-1]")

plt.tight_layout()
plt.show()

```



Problem 12

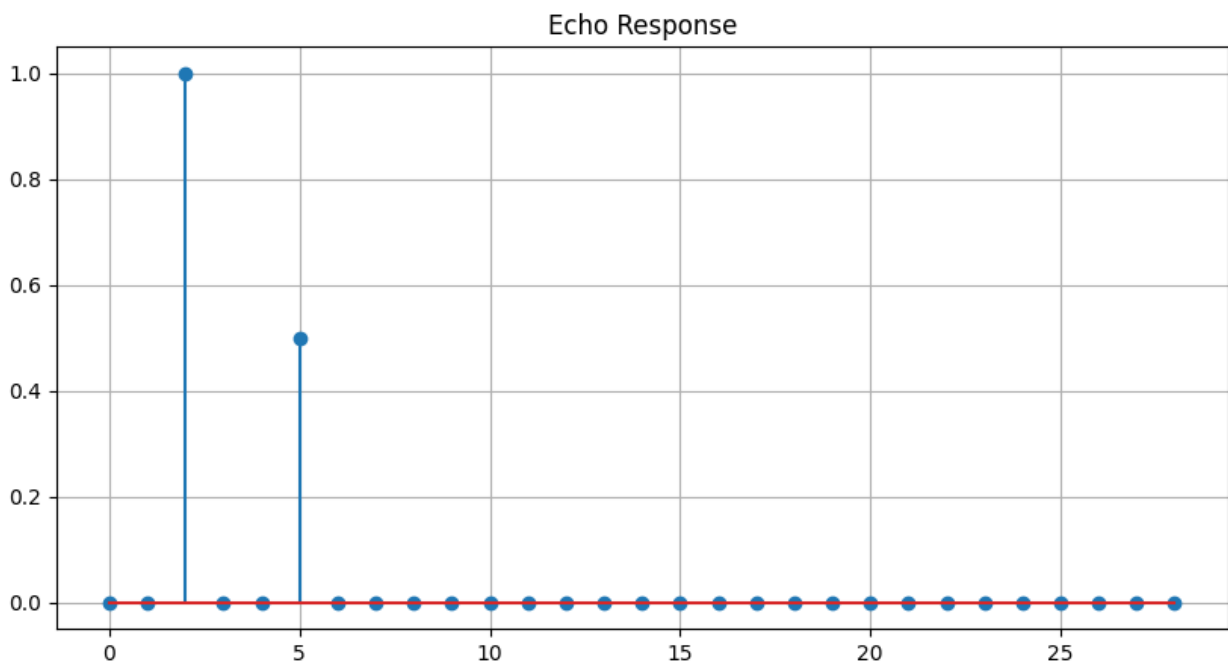
Simulate an echo system.

Impulse Response:

$$h[n] = \delta[n] + 0.5\delta[n-3]$$

Observe how delayed copies appear.

```
x = np.zeros(20)
x[2] = 1
h = np.zeros(10)
h[0] = 1
h[3] = 0.5
y = np.convolve(x,h)
plt.figure(figsize=(10,5))
plt.stem(y)
plt.title("Echo Response")
plt.grid()
plt.show()
```



The second pulse appears after a delay.

This mimics an echo effect in audio systems.

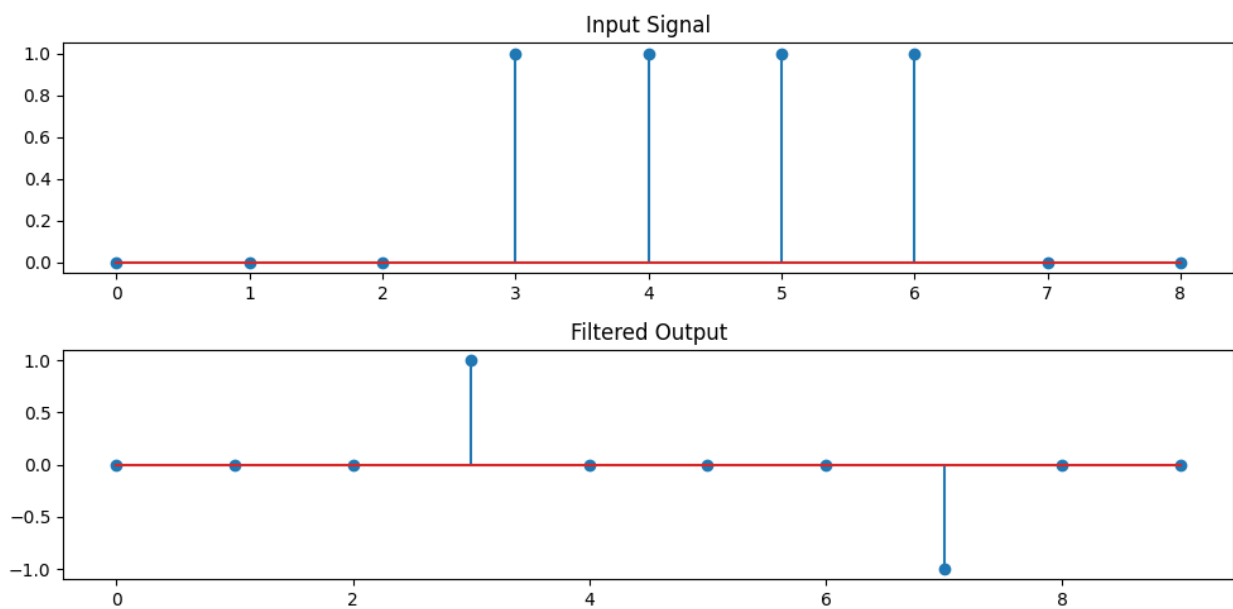
Problem 13

Apply the difference filter:

$$h[n] = [1, -1]$$

to detect abrupt changes.

```
x = np.array([0,0,0,1,1,1,1,0,0])
h = np.array([1,-1])
y = np.convolve(x,h)
plt.figure(figsize=(10,5))
plt.subplot(2,1,1)
plt.stem(x)
plt.title("Input Signal")
plt.subplot(2,1,2)
plt.stem(y)
plt.title("Filtered Output")
plt.tight_layout()
plt.show()
```



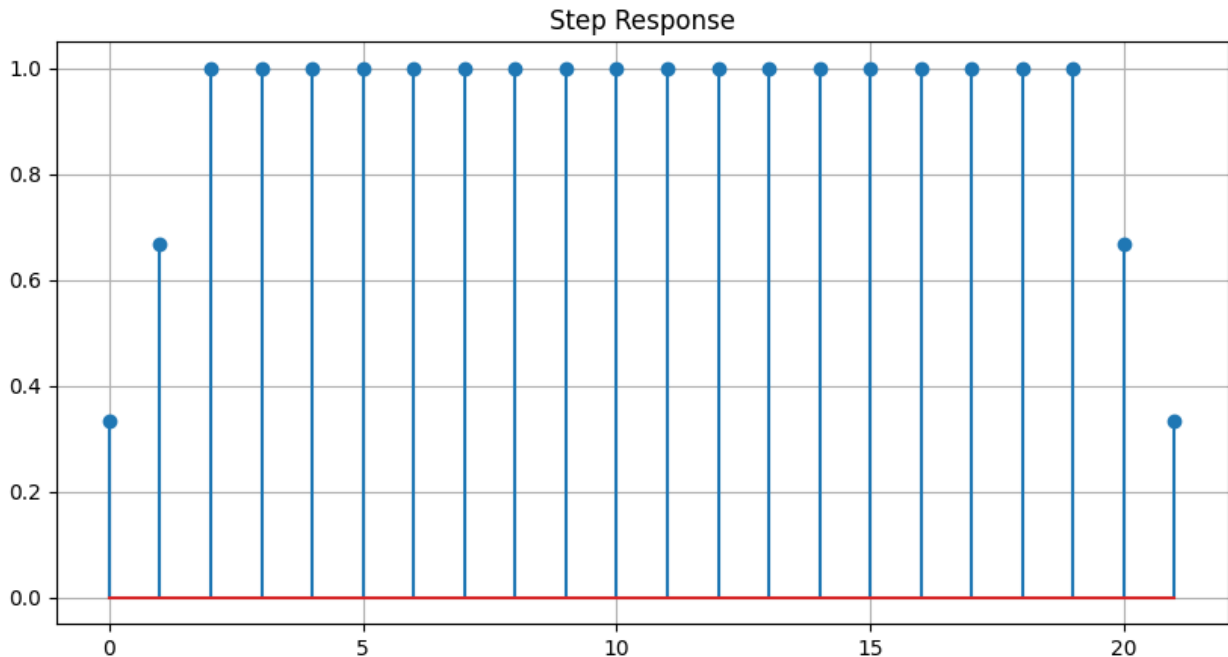
Large values occur where sudden changes are present.

This is the principle behind edge detection.

Problem 14

Determine the step response of a moving average system.

```
x = np.ones(20)
h = np.array([1,1,1])/3
y = np.convolve(x,h)
plt.figure(figsize=(10,5))
plt.stem(y)
plt.title("Step Response")
plt.grid()
plt.show()
```



The response gradually rises and then settles.

This behavior is common in practical systems.

Problem 15

Verify:

$$x * (h_1 + h_2) = x * h_1 + x * h_2$$

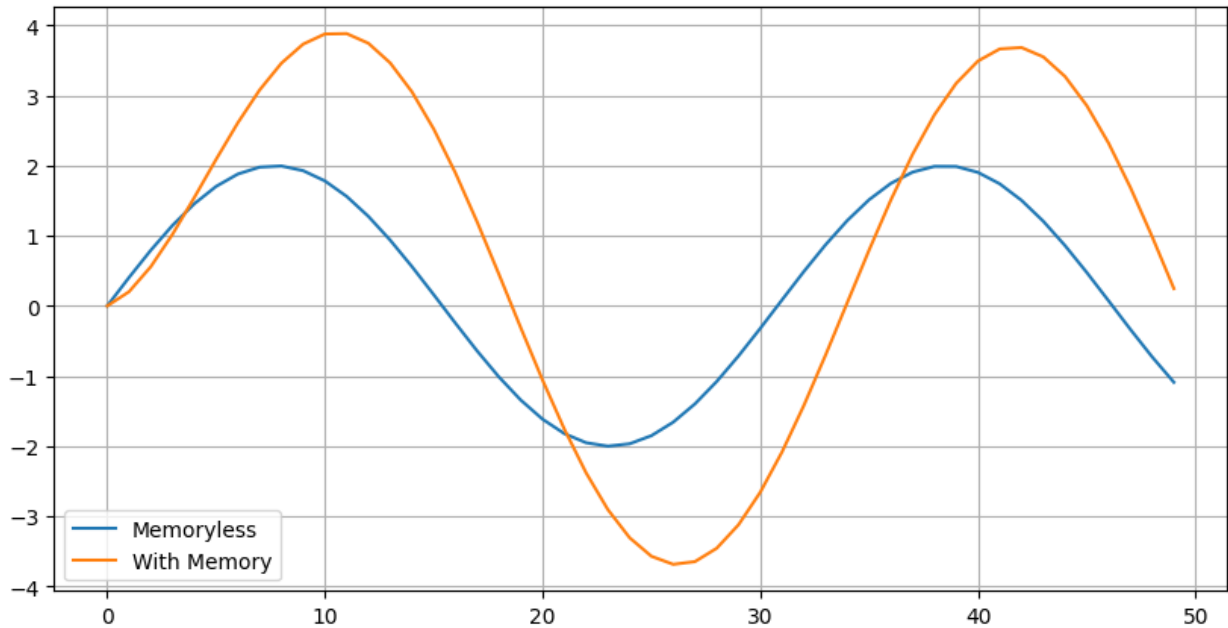
```
x = np.array([1,2])
h1 = np.array([1,1])
h2 = np.array([1,-1])
lhs = np.convolve(x,h1+h2)
rhs = np.convolve(x,h1) + np.convolve(x,h2)
print(lhs)
print(rhs)

[2 4 0]
[2 4 0]
```

Problem 16

Compare a memoryless system and a system with memory.

```
N = 50
x = np.sin(np.linspace(0,10,N))
y_memoryless = 2*x
y_memory = np.zeros(N)
for n in range(1,N):
    y_memory[n] = 0.8*y_memory[n-1] + x[n]
plt.figure(figsize=(10,5))
plt.plot(y_memoryless,label="Memoryless")
plt.plot(y_memory,label="With Memory")
plt.legend()
plt.grid()
plt.show()
```



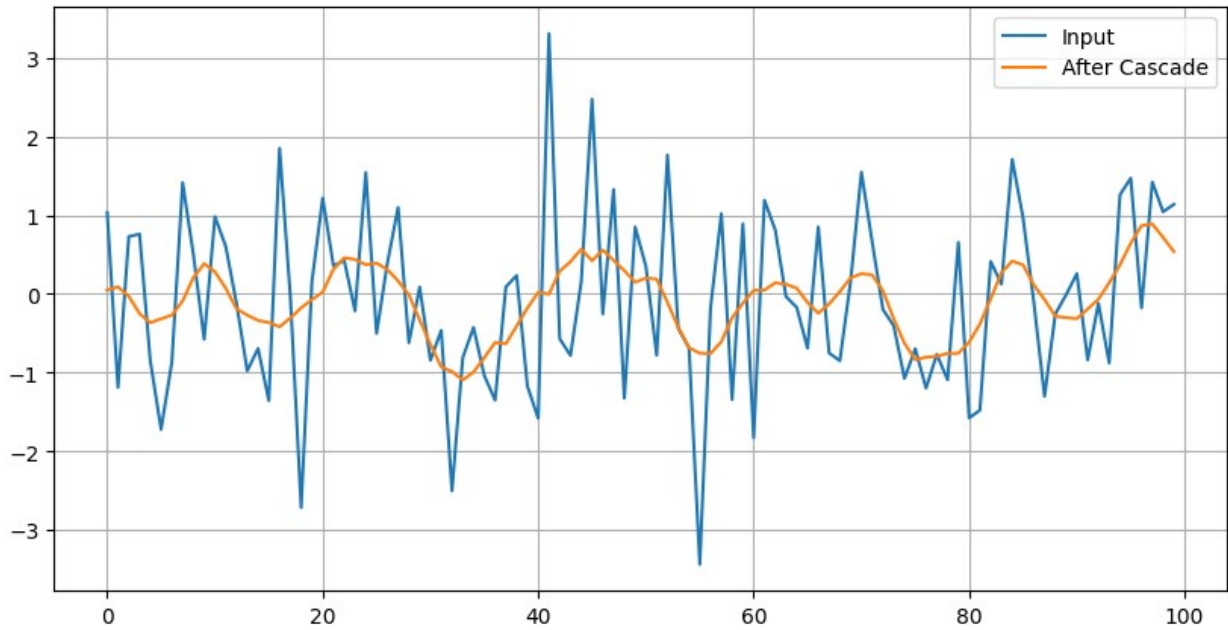
The second system depends on previous outputs.

Hence it possesses memory.

Problem 17

Investigate the effect of cascading two filters.

```
x = np.random.randn(100)
h1 = np.ones(3)/3
h2 = np.ones(5)/5
y1 = np.convolve(x,h1,mode='same')
y2 = np.convolve(y1,h2,mode='same')
plt.figure(figsize=(10,5))
plt.plot(x,label="Input")
plt.plot(y2,label="After Cascade")
plt.legend()
plt.grid()
plt.show()
```



Challenge Problem

Create a system that:

1. Delays a signal by 5 samples.
2. Produces an echo.
3. Smooths the output.

Use convolution to implement the complete system.